RISK 231: VBA

Himchan Jeong, FSA

Department of Statistics & Actuarial Science
Simon Fraser University

VBA: Intro, the VBE, and Recording Macros

What is VBA?

- "Visual Basic for Applications"
- A programming language built into Excel
  - Make sophisticated workbooks
  - Write custom functions
- What is a **macro**?
  - A collection of code that performs a series of commands
  - Very useful when doing repetitive jobs to improve efficiency
  - In Excel, macros can be written or recorded
- This website has a long list of things you can do with VBA, and tutorials:

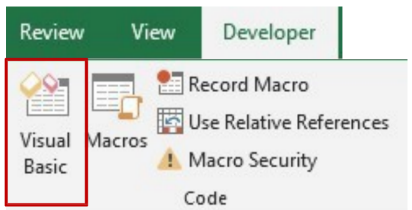
  http://www.excel-easy.com/vba.html

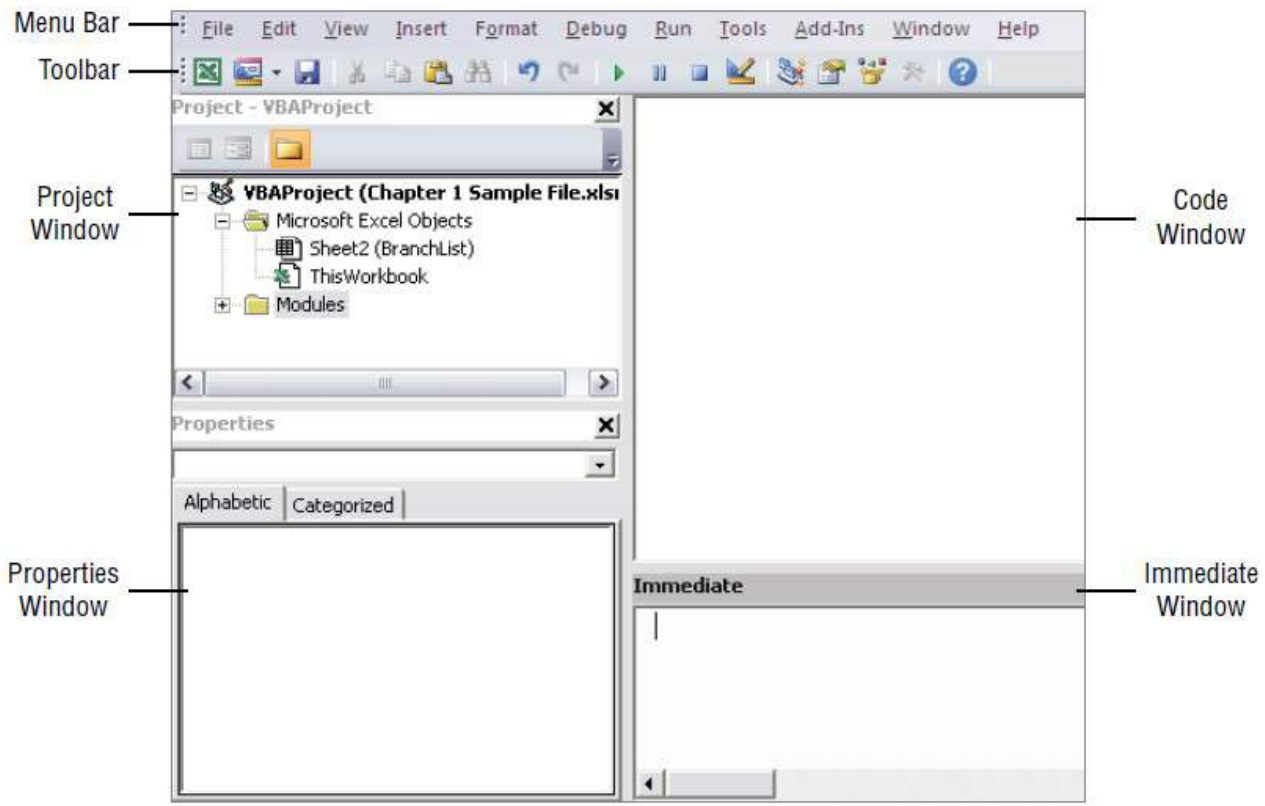
Why use VBA?

- **Automate a manual and repetitive process to save time and eliminate human error**

The Visual Basic Editor (VBE)

Open the VBE from the **Developer** tab: click **Visual Basic**.

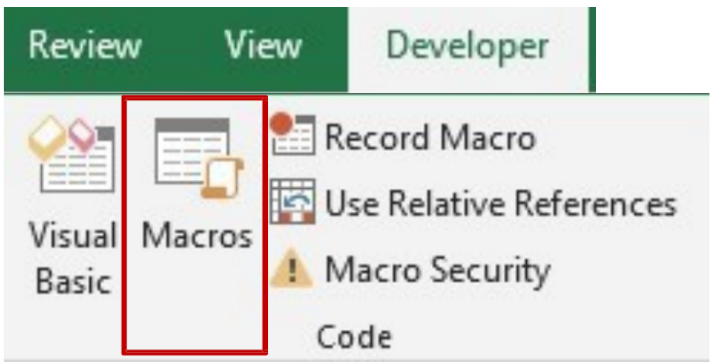




The Visual Basic Editor: getting code into a module

- Before you can do anything meaningful, you must have some VBA code in the VBA module. You can get VBA code into a VBA module in three ways:
  - **Use the Excel macro recorder to record your actions and convert them to VBA code.**
  - **Enter the code directly.**
  - **Copy the code from one module and paste it into another.**

Entering the code directly:



Name your macro (e.g. NewMacro) and click "Create".

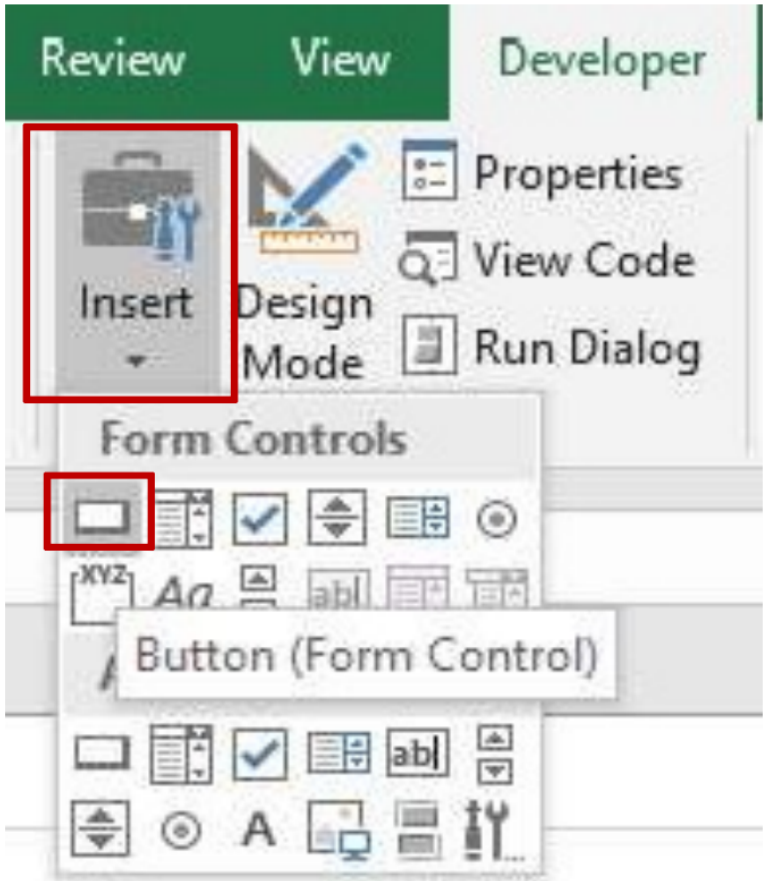
```
Sub NewMacro()

    MsgBox "Hello World"

End Sub
```

Run it with the **Run** button (▶) on the VBE toolbar (or press F5).

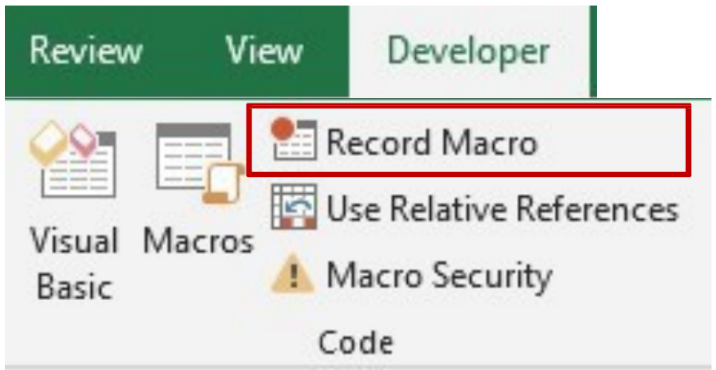
The Visual Basic Editor: buttons and other cool things...



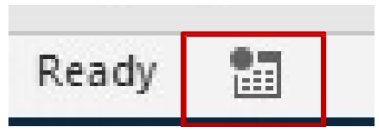
- **Developer** $\rightarrow$ **Insert** $\rightarrow$ **Form Controls** $\rightarrow$ **Button (Form Control)**.
- Put the button wherever you want and assign it to your new macro.

Recording a Macro

Using the Excel macro recorder to record your actions and convert them to VBA code:



or



- Name your new macro, put in a shortcut key (if you'd like). Do stuff, then push stop.
- You can then see the automatically written code and adjust as desired.

Some coding rules and good habits

- White space is ignored in VBA and the editor will automatically discard any extra spaces.
  - Always put a space after a comma, never before, just like in regular English.
  - Most infix operators (`=`, `+`, `-`, etc.) should always be surrounded by spaces.
  - There are a few exceptions, which should never be surrounded by spaces, for example: power (`^`), `$`, `@`.
- Indentation should be used to highlight the block structure and control statements.
  - You can use the "**Tab**" key to indent your lines of code that logically belong together to make the code easier to read.
  - The "**Tab**" key should be used instead of pressing the spacebar multiple times.

VBA: Variables and Objects

Common Variable Types

Type	Description	Examples
`Integer`	Any whole numbers from −32,768 to 32,767	0, −1, 3, −12,345, 80
`Long`	Same as Integers, but can accept a much wider range (plus or minus 2 billion)	1,849,418, −766,484
`Double`	Double-precision floating point decimal	3.14159, 0.01, 100.99, −99,481,147.33544
`String`	Contains an array of characters	“String Example1”, “SFU”
`Boolean`	True or false variable used for logic statements	TRUE, FALSE
`Variant`	Variable that can take any of the above forms (and more). Requires larger amounts of system memory	All of the above

Declaring Variables

Before a variable is used, it must be declared by using the `Dim` statement.

`Dim` {*Variable Name*} `As` {*Variable Type*}

You can name a variable anything you want, subject to a few rules:

- Must begin with a letter
- Cannot contain spaces (use an underscore instead)
- Cannot be a "reserved" name (Ex: `Sub`, `Dim`, `If`, `True`, etc.)
- Cannot contain special characters (Ex: `+`, `-`, `&`, `$`, etc.)

Variable type can be anything from the previous slide

- If you don't specify a variable type, the default is `Variant`

- Multiple variables can be declared in a single `Dim` statement via comma separation.
- If you use a variable without declaration, it will be assumed as `Variant` (NOT recommended).

Assign values to variables

To store data into a variable is called *assignment*.

[Let] {*Variable Name*} = {*Data*}

Here "Let" usually can be ignored (i.e. we usually just write {*Variable Name*} = {*Data*}).

Assign data value on the right to the variable on the left.

Example:

```
Dim MyVariable As Integer
MyVariable = 2
```

Sometimes we will call it "initialization" when we assign an initial value to a variable.

Try it: declaration and TypeName

```
Sub Test()

    Dim x, y As Integer
    z = 1
    MsgBox TypeName(x)

End Sub
```

- `TypeName(x)` returns the type of `x` as a string.
- In `Dim x, y As Integer` only `y` is an `Integer`; `x` has no type of its own, so it is a `Variant`.
- `z` is never declared, so it is also a `Variant`.

Why do variable types matter?

Run these three procedures and compare the message boxes:

```
Sub VtypeDemo1()
    Dim x as Integer
    x = 3.14
    MsgBox x
End Sub
```

```
Sub VtypeDemo2()
    Dim x as Double
    x = 3.14
    MsgBox x
End Sub
```

```
Sub VtypeDemo3()
    Dim x as Boolean
    x = 3.14
    MsgBox x
End Sub
```

Examples with Variant

To understand how Variant data types work, try executing these procedures:

```
Sub VariantDemo1()
    MyVar = "123"
    MyVar = MyVar + MyVar
    MyVar = "Answer: " & MyVar
    MsgBox MyVar
End Sub
```

```
Sub VariantDemo2()
    MyVar = 123
    MyVar = MyVar + MyVar
    MyVar = "Answer: " & MyVar
    MsgBox MyVar
End Sub
```

Examples with `Variant` (cont.)

- For the first procedure, the message box displays `Answer: 123123`. As you assigned `MyVar` as `"123"`, it is implicitly considered as a text string, so the "`+`" operator will join (concatenate) the strings together.
- For the second procedure, the message box displays `Answer: 246`. As you assigned `MyVar` as `123`, it is implicitly considered as an integer, so the "`+`" operator will sum up the two values.

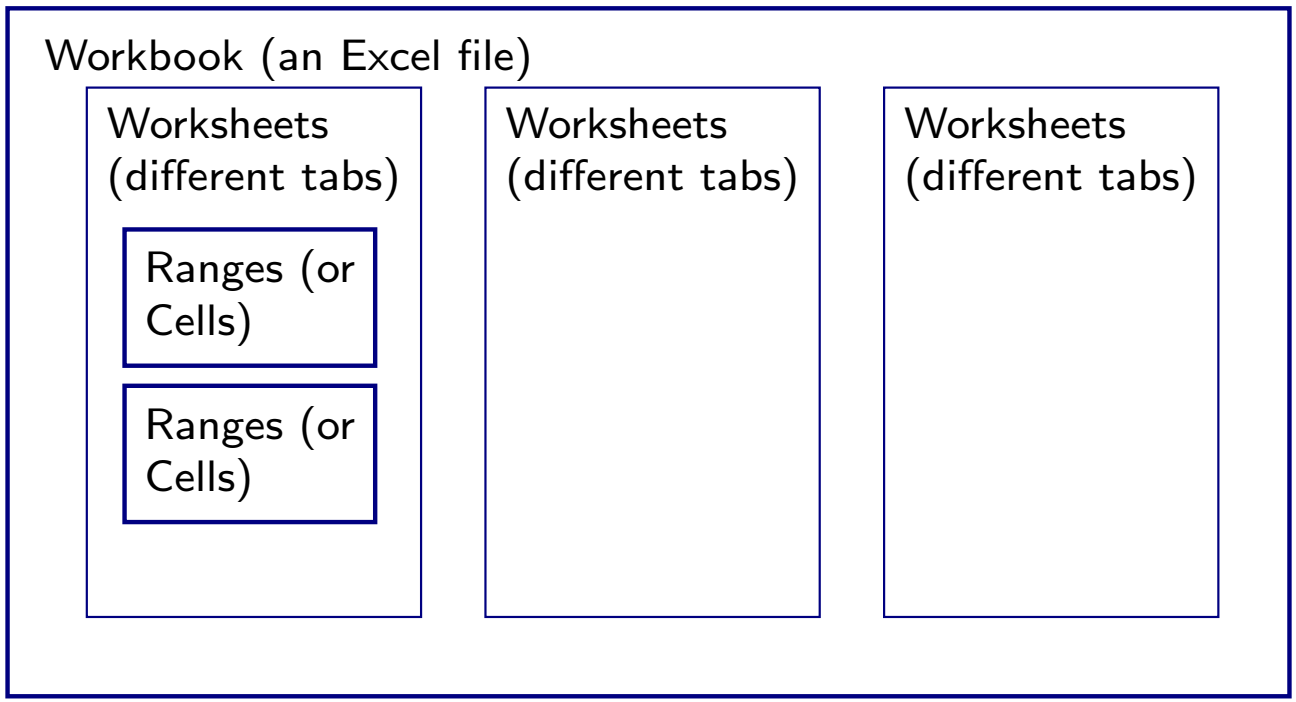
Variable Scope

A variable's *scope* determines in which modules and procedures you can use the variable.

Scope	**To Declare a Variable with This Scope**
Single (Sub) procedure	Include a `Dim` or `Static` statement within the (Sub) procedure.
Single module	Include a `Dim` or `Private` statement before the first procedure in a module.
All modules	Include a `Public` statement before the first procedure in a module.

What are Objects?

- Objects are things that we can control or manipulate in Excel by VBA code.
- Structure of objects in Excel is like:

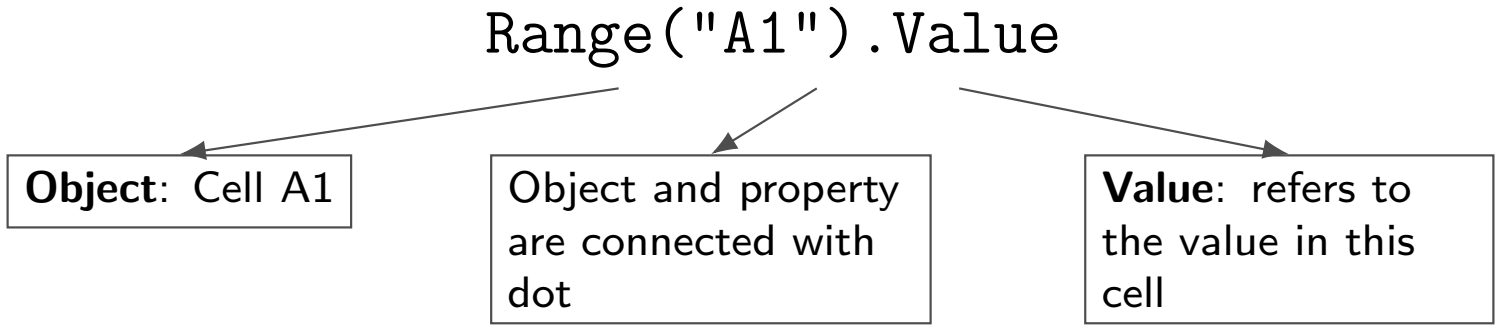


Common VBA Objects

Objects	Descriptions
`Application`	Represents the Excel software application
`Workbooks`	Represents workbooks in Excel (an Excel file)
`Worksheets`	Represents worksheets in a workbook (it is just the tabs we usually refer to)
`Sheets`	Object that includes `Charts` and `Worksheets` objects. Sometimes `Sheets` and `Worksheets` can be used interchangeably.
`Range`	Represents cells in an Excel worksheet; it can be a cell: `Range("A1")` or a range: `Range("A1:B3")`
`Cells`	Represents a single cell in an Excel worksheet, specified by row number and column number. For example: `Cells(3, 1)` is equivalent to `Range("A3")`.

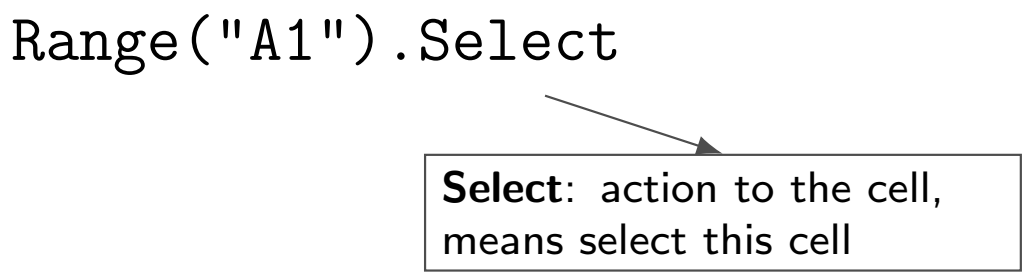
Properties and Methods of Objects

Properties are essentially the characteristics of an object.



There are some other properties like `Name`, `Rows.Count`.

Methods are the actions that can be performed with an object.



There are also some other usual methods like `Copy`, `Paste`, `ClearContents`, etc.

Understanding Objects

- To point to a specific object in VBA, you can traverse the object model. For example, to get to cell A1 on Sheet 1, you can enter this code:

```
Application.ThisWorkbook.Sheets("Sheet1").Range("A1").Select
```

- Entering this code also gets you to cell A1 because Excel infers that you mean the current workbook and the active sheet:

```
Range("A1").Select
```

- Indeed, if you have your cursor already in cell A1, you can simply use the `ActiveCell` object, negating the need to spell out the range.

```
ActiveCell.Select
```

Some Other Useful Objects

Objects	Descriptions
`ActiveCell`	The active cell.
`ActiveSheet`	The active sheet.
`ActiveWorkbook`	The active workbook.
`Selection`	The object selected. It could be a `Range` object, `Chart` object and so on.

Assign Values to Object Variables

We can also declare a variable of an object type and assign object data to it:

```
Dim {Object Variable Name} As {Object Type}
```

Can be: `Range`, `Worksheet`, `Workbook`

```
Set {Object Variable Name} = {Object Data}
```

MUST have "Set" here

Example:

```
Dim rng As Range
Set rng = Range("A1:B8")
```

InputBox

One can directly receive an input value from the user and use it as a variable in VBA via **InputBox**.

```
{Variable Name} =
Application.InputBox(prompt:="[your message]", Type:=??)
```

Value for `Type`	Description
0	A formula
1	A number
2	Text (a string)
4	A logical value (**True** or **False**)
8	A cell reference, as a **Range** object
16	An error value, such as `#N/A`
64	An array of values

MsgBox

One can display the value of a variable (along with pre-specified strings).

```
Sub MsgBoxDemo1()
    MyVar = 123
    MsgBox MyVar
End Sub
```

```
Sub MsgBoxDemo2()
    MyVar = 123
    MsgBox "My value is " & MyVar
End Sub
```

```
Sub MsgBoxDemo3()
    x = 2
    y = 4
    z = x + y
    MsgBox x & " + " & y & " = "
     & z
End Sub
```

VBA: If Statements, Loops, and Functions

If Statements – Excel

- An **IF** statement is the most commonly used control statement.
- We've learned the Excel version:

  `=IF( {condition}, {value if true}, {value if false} )`

  - {Condition} is an expression that evaluates to a Boolean (true / false) data type.
  - If the expression returns true, the {value if true} expression is returned. If false, the {value if false} expression is returned.
- The VBA version works almost the same way. There's just different notation and a little more flexibility.

If . . . Then Statements

- Below is an example of a simple `If . . . Then` statement in VBA.

```
If {Condition} Then
    {Stuff}
End If
```

- Read the statement like this:
  - **If** the condition is true, **Then** execute the commands between `Then` and **End If**. If the condition is false, then go to the line after `End If`.
- The condition piece often features logical arguments (`True`, `False`, `<`, `>`, etc.)
- Inside a condition, "=" is a test of equivalence. It is not an assignment statement.

If . . . Then . . . Else Statements

- This expanded statement is analogous to the IF function in Excel.
- Read the statement like this:
  - **If** the condition is true, **Then** execute the commands between `Then` and **Else**. If the condition is false, then execute the commands between **Else** and **End If**.

```
If {Condition} Then
    {Stuff}
Else
    {Different Stuff}
End If
```

If . . . Then . . . ElseIf Statements

- Read the statement like this:
  - **If** the condition is true, **Then** execute the commands between `Then` and **ElseIf**. If the condition is false, then . . .
  - If the second condition is true, **Then** execute the commands between the second `Then` and **End If**.

```
If {Condition} Then
    {Stuff}
ElseIf {Different Condition} Then
    {Different Stuff}
End If
```

If . . . Then . . . ElseIf . . . ElseIf . . . Else Statements

- You can create very complex and robust if statements by combining the concepts on the past few slides.

```
If {Condition} Then
    {Stuff}
ElseIf {Condition 2} Then
    {Stuff 2}
ElseIf {Condition 3} Then
    {Stuff 3}
ElseIf {Condition 4} Then
    {Stuff 4}
ElseIf {Condition 5} Then
    {Stuff 5}
ElseIf {Condition 6} Then
    {Stuff 6}
Else
    {Stuff 7}
End If
```

Nesting

- A control structure is said to be nested if it lives inside a control structure of a similar type.
- These equivalent examples highlight a nested if statement in both Excel and VBA.
- The highlighted statements are nested because they are located entirely within the confines of an existing if statement.

Excel Formula

```
=IF(A1<>0,
   IF(A1>0,LN(A1),LN(-A1)),
   0)
```

"<>" means *not equal to*.

VBA Statements

```
Function X(ByVal Y As Double) _
    As Double

    If Y <> 0 Then
        If Y > 0 Then
            X = Log(Y)
        Else
            X = Log(-Y)
        End If
    Else
        X = 0
    End If
End Function
```

Two Observations When Using Functions

1. Using parentheses to enclose arguments while using functions:

- **A simple rule**: if a function call is the ONLY statement on the line, parentheses aren't needed.

```
Sub MsgBoxDemo4()
    MsgBox "Hello World"
End Sub
```

- Otherwise (if there are other statements or elements), parentheses MUST be added to distinguish function arguments and other statements.

```
Sub MsgBoxDemo5()
    Dim ans As Integer
    ans = MsgBox("Continue?", vbYesNo)
End Sub
```

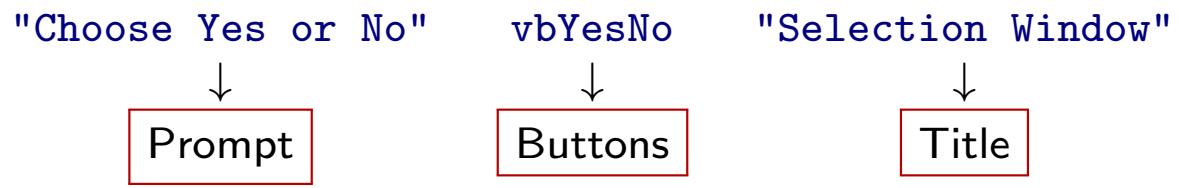
- See also: Creating Custom Message Boxes with VBA MsgBox Function (automateexcel.com)

Two Observations When Using Functions (cont.)

2. When you use a function, you can either

- supply arguments *positionally*, in the order that they appear in the procedure's definition:

```
Sub MsgBoxDemo6()
    MsgBox "Choose Yes or No", vbYesNo, "Selection Window"
End Sub
```



- or you can supply the arguments *by name* without regard to position:

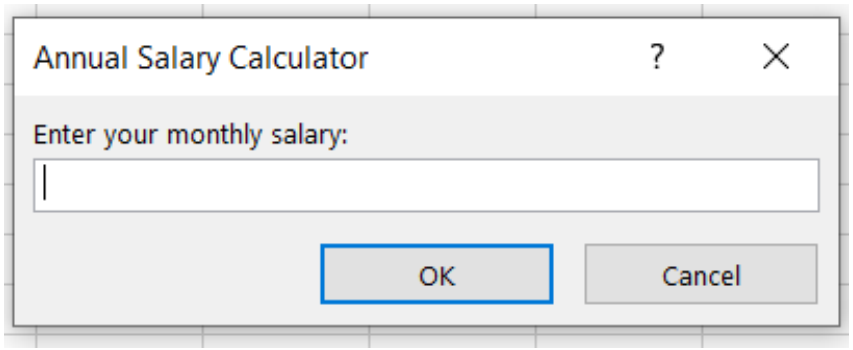
```
Sub MsgBoxDemo7()
    MsgBox prompt:="Hello World", Title:="This is a title"
End Sub
```

- A **named argument** consists of an argument name followed by a colon and an equal sign (`:=`), followed by the argument value.

InputBox Function Example

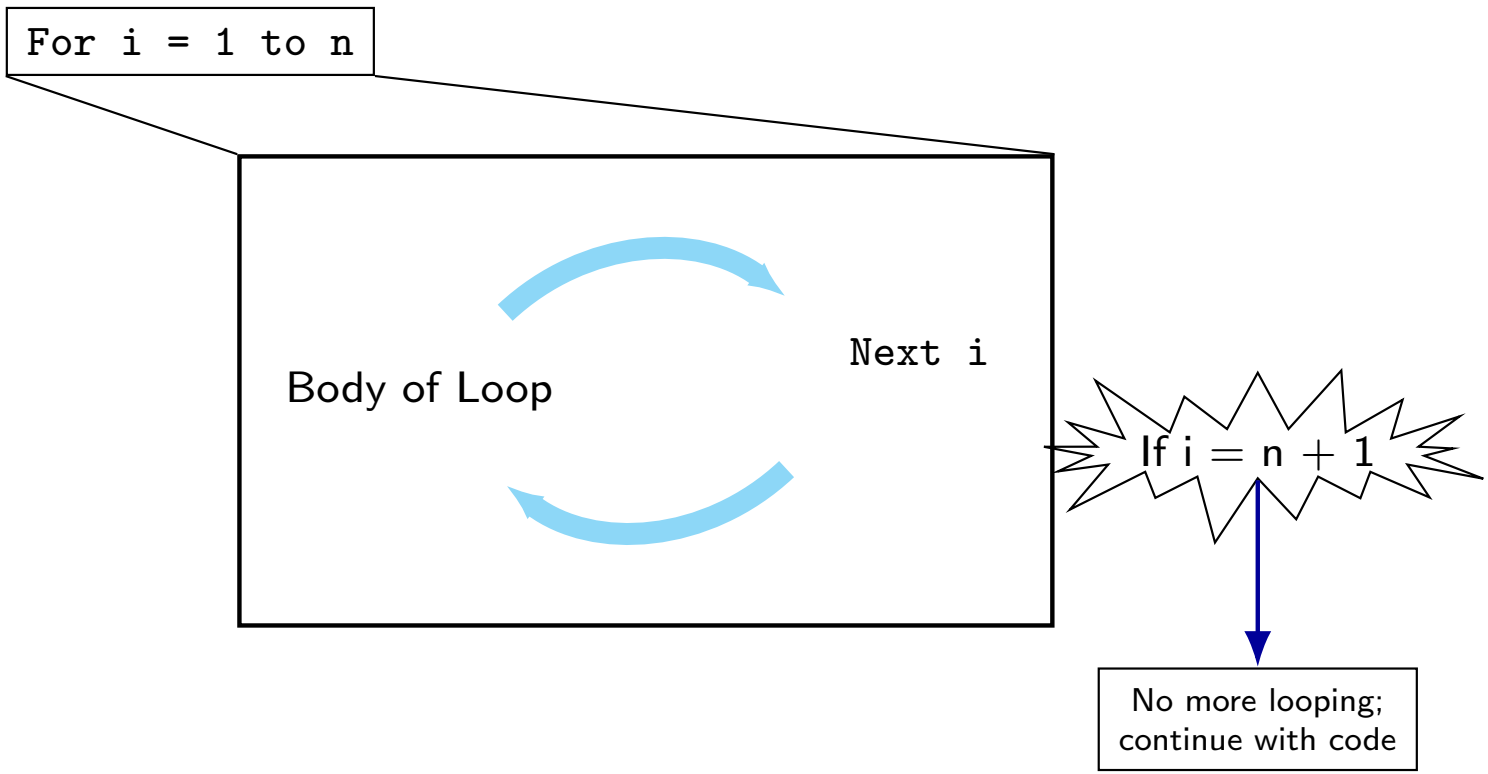
```
Sub GetValue()
    Dim Monthly As Integer
    Monthly = Application.InputBox("Enter your monthly salary:", _
        Title:="Annual Salary Calculator", Type:=1)
    MsgBox "Annualized: $" & (Monthly * 12)
End Sub
```

- `Type:=1` restricts your input to be numbers.
- The underscore _ at the end of a line continues the statement on the next line.
- The input box shows up when the code is executed:



For-Next Loops

- **For-Next Loops** are an easy, extremely useful way to repeat structurally similar tasks many times over.
- Mathematical notation: $\sum_{i=1}^{n} X_i$ $\prod_{i=1}^{n} X_i$



For-Next Loop Format

```
For {Index} = {low} To {high}
    {Body of Loop}
Next {Index}
```

Components of a For-Next Loop:

- Index – execution of the loop hinges upon the value of an index variable
  - The index should be an `Integer` or `Long` variable
  - Don't forget to declare the index as a variable first!
- Low Value – initial value an index takes
  - On first loop iteration, Index = low value

Read the statement like this: run the loop (high − low +1) times, each time increasing the value of the index by 1.

- High Value – final value an index takes
  - On final loop iteration, Index = high value
- Next – must be at the end of the loop
  - If Index < High Value, Index increases by 1 and the loop reruns (new iteration)
  - If Index = High Value, code continues to the statement below `Next`

Do Loops

- A Do Loop is another common control structure used for repeating sections of code.
- Two types:
  - Execute loop While a certain condition exists
  - Execute loop Until a certain condition exists
- Uses of a Do Loop
  - Brute force solve methods
  - Parameter calibration (e.g.)
    - Refine estimate *while* square error is greater than tolerance level
    - Refine estimate *until* square error is less than tolerance level
- Not as common as For Loops, but some notation is included in case you're interested.

```
Do While {Condition}
    {Stuff}
Loop
```

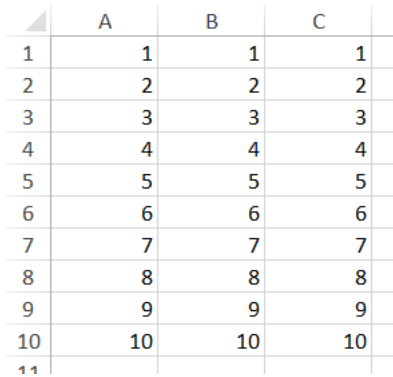
```
Do Until {Condition}
    {Stuff}
Loop
```

Some Looping Techniques

A usual application of loops is to populate values row by row. There are several ways to do it:

1. Use the `Offset` method of the `Range` object.
2. Use the `Cells` object.
3. Use the `Range` object, incorporating a concatenation trick.

```
Sub LoopDemo()
    Dim i As Integer
    For i = 1 To 10
        ' Use Offset to populate col A
        Range("A1").Offset(i - 1, 0).Value = i
        ' Use Cells to populate col B
        Cells(i, 2).Value = i
        ' Use concatenate and Range to populate col C
        Range("C" & i).Value = i
    Next i
End Sub
```



	A	B	C
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7
8	8	8	8
9	9	9	9
10	10	10	10

`Offset(RowOffset, ColOffset)`
RowOffset: how many rows to move down (negative means move up)
ColOffset: how many columns to move right (negative means move left)

Some Looping Techniques: `Range.Count`

- `Range.Count` method:
  - Returns a value that represents the number of objects (cells), no matter empty or nonempty.
  - For example, the following statement returns value 8.

```
MsgBox Range("A1:A8").Count
```

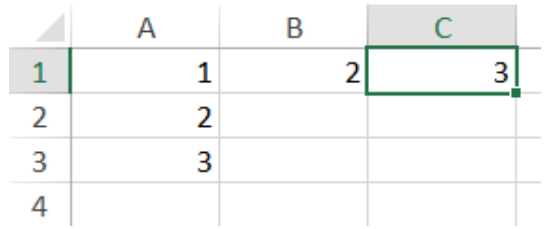
- **An important and useful technique**: how to use VBA to determine how many records are in a table?
  - This is a practical question because when you use a For-Next loop, you always want to know how many loops we should do (i.e. what is the High value?).
  - In the following example, we are going to use a combination of methods: `Count` and `End`.

Some Looping Techniques: `Range.End`

- `Range.End(`*`Direction`*`)` method:
  - Returns a `Range` object that represents the cell at the end of the region that contains the source range.
  - The argument *Direction* could be: `xlDown`, `xlToRight`, `xlToLeft` or `xlUp`.
  - For example:

```
Range("A1").End(xlToRight).Select
```

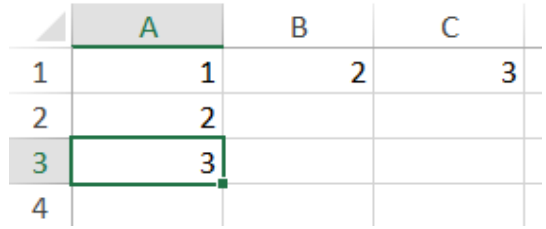




Ctrl + →

```
Range("A1").End(xlDown).Select
```





Ctrl + ↓

Some Looping Techniques: Count and End together

- Use a combination of `Count` and `End` to help us determine the number of records in a data table.
- Another expression using the `Range` object: `Range(Cell1, [Cell2])` can hold two objects. These are equivalent; both refer to the range from A1 to A5:

```
Range("A1:A5")
Range(Range("A1"), Range("A5"))
```

```
Range(Range("B3"), Range("B3").End(xlDown)).Count
```

- `Range("B3")`: start from cell B3
- `Range("B3").End(xlDown)`: to the last cell in the column that contains cell B3; the range is what Ctrl + Shift + ↓ selects
- `.Count`: count the number of cells in this range ⇒ 10 records (B3:B12)

	A	B	C
1			
2		**Student ID**	**Score**
3		A34876	93
4		A86027	77
5		A82387	85
6		A57241	96
7		A72782	94
8		A65428	80
9		A62899	72
10		A74350	98
11		A58108	91
12		A51156	88
13			